ELEC-H310 – Digital electronics
Th05: Number encodings & Boolean algebra

Dragomir Milojevic
Université libre de Bruxelles

v3.5

Today

1. Numbers representation
2. Base conversions
3. Arithmetic operations
4. Negative numbers
5. Boolean algebra

1. Numbers representation

Real (decimal) numbers representation

- Example of a real number, in base 10:

$$(1372.6450)_{10}$$

- Two parts, split with a decimal point (. or , are allowed):

1372	.	6450
Integer $10^3 10^2 10^1 10^0$	Decimal point	Decimal $10^{-1} 10^{-2} 10^{-3} 10^{-4}$
$1 \times 10^3 + 3 \times 10^2$ $+7 \times 10^1 + 2 \times 10^0$	$+$	$6 \times 10^{-1} + 4 \times 10^{-2}$ $+5 \times 10^{-3} + 0 \times 10^{-4}$

- This is a fixed point representation; used for fractional and/or non-integer part of of decimal numbers with predefined & fixed number of digits (bits) decided at circuit design time
  - ▹ Opposed to floating point representation, used for real numbers in most digital systems (processors/ALUs) these days; not covered here

General form (for any base)

- Number N in base r, noted N_r:

$$N_r = (a_n a_{n-1} a_{n-2} \ldots a_0 b_1 b_m)_r$$

where:

  - $a_0, \ldots, a_n$ – are $n+1$ digits used for integer part I
  - $b_1, \ldots, b_m$ – are m digits used for decimal part D
  - in the above different digits are concatenated

- Number N_r is then written as:

$$\begin{aligned} N_r &= a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + a_{n-2} \cdot r^{n-2} + \ldots + a_0 \cdot r^0 \\ &\quad + b_1 \cdot r^{-1} + b_2 \cdot r^{-2} + \ldots + b_m \cdot r^{-m} \\ &= \sum_{i=0}^{i=n} a_i \cdot r^i + \sum_{j=1}^{j=m} b_j \cdot r^{-j} \\ &= I + D \end{aligned}$$

Useful bases

- In these lectures we will consider useful bases
  - Base $r = 10$ – because of us, humans, and
  - Bases $r = 2$ – binary, $r = 8$ – octal and $r = 16$ – hexadecimal
    - These 3 are used in digital systems and computers, they are power of 2 ($2^1 \rightarrow$ binary, $2^3 \rightarrow$ octal, $2^4 \rightarrow$ hexadecimal)
- Each base defines a set of symbols (digits) $\mathtt{D}$ used to describe numbers using positional notation:
  - decimal $\mathtt{D} \in \{\mathtt{0,1,2,3,4,5,6,7,8,9}\}$
  - binary $\mathtt{D} \in \{\mathtt{0,1}\}$
  - octal $\mathtt{D} \in \{\mathtt{0,1,2,3,4,5,6,7}\}$
  - hexadecimal $\mathtt{D} \in \{\mathtt{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F}\}$
- It is important to learn how to switch from one base to another – conversions, covered in the next Chapter; and how to perform basic arithmetic computations (Chapter 3 of this lecture)

General form (for any base) 2/2

- Looking at the integer part of the number N in base r:

$$N_r = \sum_{i=0}^{i=n} a_i \cdot r^i + \sum_{j=1}^{j=m} b_j \cdot r^{-j} = I + D$$

- We can define:
  - Least Significant Digit for $i = 0$
  - Most Significant Digit for $i = n$
  - For binary base, the above becomes Least Significant Bit (LSB) and Most Significant Bit (MSB)
- Knowing if the word starts with the MSB or LSB defines the **direction** in which we should read that word; in these lectures most of the time MSB is on the left and LSB is on the right (unless specified otherwise)
  - If you are not sure, ask, because $(1011)_2$ could be interpreted as 11 or 13 and this is obviously very much different

Numbers in different (useful) bases

Decimal (10)	Binary(2)	Octal (8)	Hexadecimal (16)
0	00000	0	0
1	00001	1	1
2	00010	2	2
3	00011	3	3
4	00100	4	4
5	00101	5	5
6	00110	6	6
7	00111	7	7
8	01000	10	8
9	01001	11	9
10	01010	12	A
11	01011	13	B
12	01100	14	C
13	01101	15	D
14	01110	16	E
15	01111	17	F
16	10000	20	10

2. Base conversions

Method of successive divisions – Dec2Bin, integer

- Number N (base 2), coded with 4 digits (integer part) written as:

$$
\begin{aligned}
N &= \sum_{i=0}^{i=3} a_i \cdot r^i \\
N &= 1 \cdot a_0 + 2 \cdot a_1 + 2 \cdot (2 \cdot a_2) + 2 \cdot (2 \cdot (2 \cdot a_3)) \\
N &= 2 \cdot (2 \cdot (2 \cdot (a_3) + a_2) + a_1) + a_0
\end{aligned}
$$

- Last expression can be reorganized and written as:

$$
\begin{aligned}
(N - a_0) : 2 &= 2 \cdot (2 \cdot (a_3) + a_2) + a_1 && (1) \\
((N - a_0) : 2 - a_1) : 2 &= 2 \cdot (a_3) + a_2 && (2) \\
(((N - a_0) : 2 - a_1) : 2) - a_2) : 2 &= a_3 && (3)
\end{aligned}
$$

- Eq. (1) gives us LSB, (2) next bit and (3) the last bit or MSB

Example Dec2Bin, integer part

Convert following number from base 10 to base 2: $(245)_{10} = (?)_2$

- Build a table: each line represents one division, so one result bit

Computation	Decimal	Base	Remainder	Read direction
$\vert$	245	:2	1	$\uparrow$
$\vert$	122	:2	0	$\vert$
$\vert$	61	:2	1	$\vert$
$\vert$	30	:2	0	$\vert$
$\vert$	15	:2	1	$\vert$
$\vert$	7	:2	1	$\vert$
$\vert$	3	:2	1	$\vert$
$\vert$	1	:2	1	$\vert$
$\downarrow$	0			$\vert$

- Note the **computation direction** (from top to bottom); result is read from bottom to top $\rightarrow$ **read direction**; this is because the first division gave LSB and last MSB
- Result is written as: $(245)_{10} = (11110101)_2$

Successive multiplications – Dec2Bin, real part

- Same reasoning applied for decimal part but instead of divisions we should use multiplications; example: $(0.345)_{10} = (?)_2$

Computation	Decimal	Base	Remainder	Direction
|	.345	x2	0	|
|	.690	x2	1	|
|	.380	x2	0	|
|	.760	x2	1	|
|	.520	x2	1	|
|	.040	x2	0	|
|	.080	x2	0	|
|	.160	x2	0	|
|	.320	x2	0	|
|	.640	x2	1	|
↓	.280	x2	0	↓

- When do we stop will depend on the number of digits that we want to accept; given by fixed point and number of bits reserved
- Note result reading direction, so for 10 digits: $(.0101100001)_2$

Conversion Bin2Dec, integer & decimal part

- Conversions in that direction are straightforward
- Convert the integer part $(\texttt{11110101})_2 = (?)_{10}$

$$
\begin{aligned}
&= 1\times 2^7 + 1\times 2^6 + 1\times 2^5 + 1\times 2^4 + 0\times 2^3 + 1\times 2^2 + 0\times 2^1 + 1\times 2^0 \\
&= 128 + 64 + 32 + 16 + 0 + 4 + 0 + 1 \\
&= 245
\end{aligned}
$$

- Convert the decimal part $(.\texttt{0101100001}\ldots)_2 = (?)_{10}$

$$
\begin{aligned}
&= 0\times 2^{-1} + 1\times 2^{-2} + 0\times 2^{-3} + 1\times 2^{-4} + 1\times 2^{-5} + 0\times 2^{-6} \\
&\quad +0\times 2^{-7} + 0\times 2^{-8} + \ldots \\
&= 0\times 1/2 + 1\times 1/4 + 0\times 1/8 + 1\times 1/16 + 1\times 1/32 + 0\times 1/64 + \ldots \\
&= (8+2+1)/32 = 11/32 = 0.34375
\end{aligned}
$$

Conversion from Dec2Oct (and back)

- Successive divisions/multipl. method can be applied to any base!
- Convert $(245)_{10} = (?)_8$

Computation	Decimal	Base	Remainder	Direction
$\vert$	245	:8	5	$\uparrow$
$\vert$	30	:8	6	$\vert$
$\vert$	3	:8	3	$\vert$
$\downarrow$	0			$\vert$

so, the result is $(245)_{10} = (365)_8$

- Converting back to decimal from octal is easy:

$$\begin{aligned}(365)_8 &= 5 \times 8^0 + 6 \times 8^1 + 3 \times 8^2 \\ &= 5 + 48 + 192 \\ &= (245)_{10}\end{aligned}$$

Conversion from Dec2Hex (and back)

- Perform the following conversion: $(245)_{10} = (?)_{16}$

Computation	Decimal	Base	Remainder	Lecture
$\vert$	245	:16	5	$\uparrow$
$\vert$	51	:16	F	$\vert$
$\downarrow$	0			$\vert$

so, the result is: $(\mathtt{245})_{\mathtt{10}} = (\mathtt{F5})_{\mathtt{16}}$

- Conversion back from hex to dec (another example):

$$
\begin{aligned}
(A1F.1C)_{16} &= (?)_{10} \\
&= A \times 16^2 + 1 \times 16^1 + F \times 16^0 + 1 \times 16^{-1} + C \times 16^{-2} \\
&= 10 \times 16^2 + 1 \times 16^1 + 15 \times 16^0 + 1 \times 16^{-1} + 12 \times 16^{-2} \\
&= 2560 + 16 + 15 + 28/256 \\
&= 2591.1093...
\end{aligned}
$$

- OK but cumbersome; luckily for bases that are power of 2, there is a much easier way, with (almost) no computational hustle

Conversion technique for power 2 bases

- For power 2 bases (binary, octal, hex), conversions can be done by grouping binary digits into packs of 3 bits for octal, and 4 bits for hexadecimal, starting from LSB (this is very important)
- Remember also that any number can be completed with an arbitrary number of '0"s to the left of the MSD (MSB for binary)
- Conversions – binary (left) & hex (right) to other power 2 bases:

Base 2	011110101		
3 bits bins	011	110	101
Base 8	3	6	5
4 bits bins	0000	1111	0101
Base 16	0	F	5

Base 16	1F2		
Base 2	0001	1111	0010
3 bits bins	111	110	010
Base 8	7	6	2
Base 10	498		

Example of conversion from dec to bin, oct & hex

Perform the following conversions: $(378)_{10} = (?)_2 = (?)_8 = (?)_{16}$

- First convert from dec to bin: $(378)_{10} = (101111010)_2$
  - We need 9 bits, so for oct we do not need to append any 0's to the left; for hex we need to add 3 bits set to 0 (12 bits or 3 hex digits)
- Then from bin to oct and hex:

Base 2	101111010		
3 bits bins	101	111	010
Base 8	5	7	2
4 bits bins	0001	0111	1010
Base 16	1	7	A

- Final results is then written as:
  $(378)_{10} = (101111010)_2 = (572)_8 = (17A)_{16}$

Arbitrary conversions

- How to convert a number N in base p to a number X in base q?

$$N_p \rightarrow (X)_q$$

  It can be done using the method of successive divisions for the integer part and multiplications for the decimal part; but this could be confusing & error prone; you will be doing do it during the exercises, but not for the exam $\rightarrow$ we adopt simplified way

- Simply perform intermediate conversion to base 10 and then move to other base:

$$N_p \rightarrow N_{10} \rightarrow (X)_q$$

- For useful basis start with base 2 first, the others will follow

$$N_{10} \rightarrow (?)_2 \rightarrow (?)_8 \rightarrow (?)_{10}$$

Example of arbitrary conversion

Perform conversion from base 8 to base 5: $(25.34)_8 = (?)_5$

- First convert base 8 to base 10:

$$(25.34)_8 = (?)_{10} = 2 \times 8 + 5 \times 1 + 3 \times 8^{-1} + 4 \times 8^{-2} = (21.4375)_{10}$$

- Then convert from base 10 to base 5: $(21.4375)_{10} = (?)_5$

21	:5	1	↑	→	|	.4375	x5	2.1875	2
4	:5	4	|		|	.1875	x5	0.9375	0
0			|		|	.9375	x5	4.6875	4
			|		|	.6875	x5	3.4375	3
			|		|	.4375	x5	2.1875	2
			|		↓	.1875	x5	0.9375	0

- Result: $(21.4375)_{10} \rightarrow (41.204320)_5$

3. Arithmetic operations

Binary addition of two 1-bit words

- Assume two 1-bit words a, b; addition of these two words $s = a + b$ will have 2 bits of the result $(s_1 s_0)$ because:

$$(1)_{10} + (1)_{10} = (2)_{10} = (10)_2$$

- For all combinations of a,b we can compute the sum s that is a two bit word: sum bit (s_0) & carry bit (s_1)

a	b	s_1	s_0
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

- Above specification can be translated into a logic circuit called half-adder; we will learn later how to derive the logic equations

Binary addition of three 1-bit words

- Previous circuit is extended with an extra bit to calculate:

$$s = a + b + c$$

- Result uses a 2 bit word:

$$\begin{aligned}(1)_{10} + (1)_{10} + (1)_{10} &= (3)_{10} \\ &= (11)_2\end{aligned}$$

a	b	c	s_1	s_0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- For all combinations of a, b and c we can compute the sum s represented in the table
- Note that the bit s_1 is still called carry bit
- Above specification can be translated into a logic circuit called full-adder, another important circuit

Binary addition of 2 words a, b of n-bits

- The sum s of two words of arbitrary length (n-bits) can be done using series of additions starting from the LSB
- Result, word s is encoded with $n+1$ bits ($2 \times 2^n = 2^{n+1}$)

Carry	c_{n-1}	c_{n-2}	$\dots$	c_1	c_0	
A		a_{n-1}	$\dots$	a_2	a_1	a_0
B		b_{n-1}	$\dots$	b_2	b_1	b_0
S		s_{n-1}	$\dots$	s_2	s_1	s_0

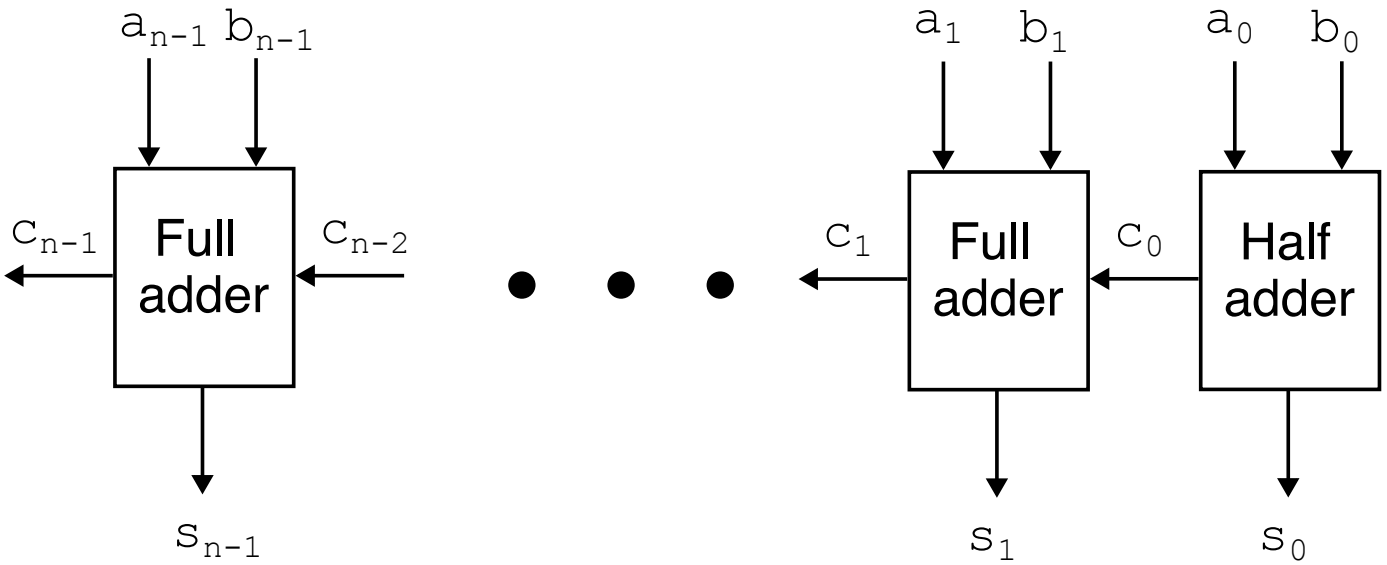
- With:

$$\begin{aligned} c_0 s_0 &= a_0 + b_0 \\ c_i s_i &= a_i + b_i + c_{i-1} \quad \forall i \in \{1, \dots, n-1\} \end{aligned}$$

Note that computation $i+1$ must wait for i to complete, due to serial nature of computation!

Addition circuit for 2 words of n bits

- LSB sum can be computed using one half-adder circuit using LSBs of a, b as operands
- Other $n-1$ bits can be computed using $n-1$ full adders $\rightarrow$ n circuits are assembled in series for a complete n-bit adder
- This circuit is known as ripple carry adder, carry needs to propagate from LSB to MSB



Above circuit will not be good for $n >>$!

Can you explain why?

Addition example with two 8-bits words

Compute $s = a + b = 236 + 170 = ?$

- We perform this operation bit-by-bit assuming two 8-bit words

Bit position (p)	8	7	6	5	4	3	2	1	0
Bit weight (2^p)	256	128	64	32	16	8	4	2	1
Carry bit	1	1	1	0	1	0	0	0	
a		1	1	1	0	1	1	0	0
b		1	0	1	0	1	0	1	0
Sum ($s_7 \ldots s_0$)	1	1	0	0	1	0	1	1	0

- Result $(406)_{10}$ can be easily verified $(406)_{10} = (1\ 1001\ 0110)_2$
- Result needs 9 bits; in systems with fixed word size (e.g., 8-bits) this will cause an overflow (bit weight 8); if the result is to be stored in an 8-bits register, memory, etc. it will be truncated
  - ▹ What will happen if this occurs in a desktop/embedded system?

4. Negative numbers

Overview

- Representation – here we cover the following forms for the representation of negative numbers:
  a. Sign and Magnitude (SM)
  b. One's complement (C1)
  c. Two's complement (C2)
- Conversions – we will look into methods that allow us to convert numbers to and from any of the above (base 10 only)
- Basic arithmetic operations – here we will discuss only C2 in details, since this is what every single computing machine is using today, the others forms are mentioned for information only

a. Signed & Magnitude (SM) representation

- Out of n bits used to represent a number is SM representation, one bit is reserved for the sign
- By convention we say:
  - 0 – for positive numbers
  - 1 – for negative numbers
  - $n-1$ bits are reserved for the magnitude
- Example of representation using one word of 8 bits:
  - We have 1 bit for the sign
  - There are 7 bits for magnitude; so 2^7 values
  - Maximum is $2^7-1=127$, minimum is -127 & therefore we can represent from `-127` to `+127` since:

```
1 1111111 (-127)      0 1111111 (+127)
```

Attention! So we have $2 \times 127 + 1 = 255$ while $2^8 = 256$

How come?

a. SM representation has 2 zeros

One '0' with $(+)$ and the other '0' with $(-)$ sign

Sign	Decimal	Binary	SVA
$+$	0	0 0000000	$(+)0$
$+$	1	0 0000001	1
$+$	...	...	
$+$	127	0 1111111	$+127$
$-$	128	1 0000000	$(-)0$
$-$	129	1 0000001	-1
$-$	...	...	
$-$	255	1 1111111	-127

a. Arithmetics using SM – subtraction

- We need to compare both signs to decide how to proceed, and we need to compare the magnitude to decide on computation direction (and the result sign)

  Example:

  - A, B are of the same $(+)$ positive sign, and
  - if $A > B$, then $A - B$, or
  - it $A < B$, then $B - A$

- SM advantages – easy to understand, we normally use this mode of representation with base 10 (negative sign before the number); very easy to convert to and from
- SM disadvantages – there are 2 different symbols with the same meaning; computations are more complex and slow since we need the adjustments that require special circuits, physical implementation (HW) is more expensive, more power, possibly lower performance

b. One's complement (C1)

- Motivation – compute subtraction like addition
- Same circuit could do two different operations (addition & subtraction) so nice optimization for HW; & works for any base
- Assume two words A & B in base $r = 10$ encoded using m digits:

$$\begin{aligned} A - B &= A + (-B) \\ &= A + (r^m - B) = A + \overline{B} \end{aligned}$$

- In the above $\overline{B}$ is the complement to base 10 (or radix complement) and $B + \overline{B} = r^m \rightarrow$ one's complement (C1)
- Practical example of computation for $r = 10, m = 3, r^m = 1000$

```
+87 --->     87              --->    87
-53 --->  1000-53=947        --->   947
-------------------------------------
                                   1034
```

b. Two's complement (C2)

- To do $A - B$ we still need to compute $\overline{B} = (r^m - B)$, so we need a subtraction operation that we wanted to get rid of
- Luckily we can re-organize B by adding and subtracting 1:

$$\overline{B} = (r^m - B) = ((r^m - 1) - B) + 1$$

where $(r^m - 1) - B$ is a complement of each digit of B noted:

$$\begin{aligned}(r^m - 1) - B &= ((r-1)(r-1)\ldots(r-1)) - (b_{m-1}b_{m-2}\ldots b_0) \\ &= ((r-1) - b_{m-1})((r-1) - b_{m-2})\ldots((r-1) - b_0) \\ &= b'_{m-1}b'_{m-2}\ldots b'_0\end{aligned}$$

  - ▹ Note that in the above, terms on the right are concatenation and not multiplication! b'_i are complement of each digit
- Complement of each digit in base 2 is simple to implement in HW: each digit is pushed through an inverter, basic logic component

b. Summary – two variants of the complement

a) One's complement (C1)

$$(r^m - 1) - B$$

Decimal	Binary	C1
0	00000000	(+)0
1	00000001	1
...	...	...
127	01111111	+127
128	10000000	−127
129	10000001	−126
...	...	...
255	11111111	(−)0

We still have 2 zeros:
00000000 &
11111111

b) Two's complement (C2)

$$(r^m - 1) - B + 1$$

Decimal	Binary	C2
0	00000000	0
1	00000001	1
...	...	...
127	01111111	+127
128	10000000	−128
129	10000001	−127
...	...	...
255	11111111	−1

There is a skew,
but only one zero!

Practical conversions to SM, C1 & C2

- Absolute value of the number is first converted to binary
  - ▹ You should use normal base 10 to 2 conversion (methods of successive divisions/multiplications)
- Note that you should complete the number with '0's to get the desired length of the word: imagine that you have to encode a word using m bits and you get n bits from the conversion, then $m-n$ '0's are placed in front of the MSB
- For SM you should keep $m-1$ bits and use bit weight m to encode the sign (0 for positive numbers, 1 for negative)
- If the number is negative, every bit of the word encoded in m bits, is inverted to get One's complement (C1); then we add 1 to C1 and derive Two's complement (C2)
- If the number is positive, just rewrite using m bits

Example of conversion to SM, C1 & C2

- Convert $(57)_{10}$ & $(-23)_{10}$ to SM, C1 & C2 using 8 bits
- For positive numbers (57_{10}) → just use binary encoding!; note completing with '0's

Absolute value	SM (8-bits)	C1 (8-bits)	C2 (8-bits)
11_1001	0011_1001	0011_1001	0011_1001

- For negative numbers (-23) → convert to binary, add sign bit for SM, invert for C1 & add 1 for C2

Absolute value	SM (8-bits)	C1 (8-bits)	C2 (8-bits)
1_0111	1_001_0111	0001_0111	
		1110_1000	1110_1001

Arithmetics with negative numbers

- We have seen that we can do arithmetics using SM & C1
  - We will not go deeper into these here $\rightarrow$ too many particular cases to analyse in order to decide on how to proceed
- However in two's complement (C2) arithmetics is straightforward!
- Imagine that you want to compute $C = A - B$
  - Positive number A is converted into C2 (just binary conversion)
  - Negative number B is converted into C2; absolute value is converted to binary, we apply bit-wise inversion, & add 1 to the C1
  - Perform subtraction as an addition
  - Check your result for the overflow with the following rule – overflow bit is ignored as long as
    - Result C is in the range: $-r^m < C < r^{m-1}$
      for 8 bits this is : $-2^7 < C < 2^7 - 1$, so: $-128 < C < 127$
    - Last two bits have the same value

Example of subtraction using C2

- Calculate using C2 the following:
  $C = A - B = 57 - 23 = ?$
- We convert and do the addition of a negative number:

	Overflow	Bits 7-4	Bits 3-0	Decimal
Carry	1	1111	001–	
A		0011	1001	$+(57)_{10}$
B		1110	1001	$-(23)_{10}$
C	1	0010	0010	$+(34)_{10}$

- Check the result:
  - Result is OK, we can ignore the overflow (see below)
  - Two last bits (MSB) of the carry are the same (here 11)
  - The result is in the range $-128 < 34 < 127$

5. Boolean algebra

Definition of Boolean algebra

- Boolean algebra is a quadruplet $\{B, ', \cdot, +\}$ where:
  - $B \rightarrow$ is a two-element set
  - $'$, $\cdot$ & $+$ are respectively $\rightarrow$ complement[1], `AND` and `OR` operators
- Depending on how we define the set B, and the 3 operators, we can define multiple Boolean algebras
- Here we focus on two valued Boolean algebra introduced & formalized by C. Shannon (1938) as a follow-up of the work from Edward V. Huntington (1904)
  - Set $B = \{0, 1\}$ is defined with:
    - 0 – `FALSE` and 1 – `TRUE`
  - And two binary operators:
    - $+$ inclusive or (`OR`) and $\cdot$ (`AND`)
  - Attention!!! – In Boolean algebra operators $\cdot$ & $+$ are not arithmetic, but logical operators, so $1 + 1 \neq 2$

[1] Also represented with a horizontal line above the variable symbol, easier for pen & paper.

Boolean algebra: definition of AND/OR operators

- Boolean operators are defined using Truth Tables $\rightarrow$ they indicate operator result for all possible combinations of arguments
- We examine all possible combinations for arguments x & y: table will contain 4 lines (2^2), and each operator has it's own column

x	y	$x \cdot y$	$x + y$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

- Note that we can manufacture electronic device to calculate the above $\rightarrow$ logic gate for which the arguments appear as inputs, and Boolean function evaluation as output

Six axioms of Boolean algebra

- For a quadruplet $\{B, ', \cdot, +\}$, to be a Boolean algebra, following axioms needs to be satisfied:
  - Axiom 1. Closure $\rightarrow$ B is closed for $+$ and for $\cdot$
    - In another words, result of operations is in B
  - Axiom 2. Neutral element $\rightarrow$ B has a neutral element for $+$ (noted '0') and for $\cdot$ (noted '1')
  - Axiom 3. Commutativity $\rightarrow$ is valid for $+$ and $\cdot$
  - Axiom 4. Distributivity $\rightarrow$ $\cdot$ distributes $+$ and $+$ distributes $\cdot$
  - Axiom 5. Complement $\rightarrow$ of x
  - Axiom 6. In B there are 2 elements x, y such that $x \neq y$
- Associativity is the consequence of the previous:

$$\begin{aligned} (a+b)+c &= a+(b+c) \\ (a \cdot b) \cdot c &= a \cdot (b \cdot c) \end{aligned}$$

- It is possible to verify 6 axioms for two-valued Boolean algebra

Basic theorems of Boolean algebra

Th1: Idempotency for $+$ & $\cdot$

$$x + x = x \text{ and } x \cdot x = x$$

Th2: Operations with neutral element

$$x + 1 = 1 \text{ and } x \cdot 0 = 0$$

Th3: Absorption

$$x \cdot (x + y) = x$$

Th4: Involution

$$(x')' = x$$

Th5: Associativity

$$(x + y) + z = x + (y + z) \text{ and } (x \cdot y) \cdot z = x \cdot (y \cdot z)$$

Th6: De Morgan's laws

$$(x + y)' = x' \cdot y' \text{ and } (x \cdot y)' = x' + y'$$

Th7: Consensus

$$x \cdot y + x' \cdot z + y \cdot z = x \cdot y + x' \cdot z$$

Proving propositions in Boolean algebra

- Proving Boolean propositions – formal demonstration of the proposition correction
- Here we use:
  - a) Axioms, theorems – use already proved results to prove new ones
    - We will use this method for exercises as illustration; it is a very good way to integrate different Axioms and Theorems of Boolean algebra (very important!!!); not required as such for the exam
  - b) Truth Tables
    - We can do this since the number of all possible combinations for Boolean arguments is limited and manageable for small number of arguments
  - c) Duality principle
    - Handy way to double the number of proven propositions

a) Proofs using axioms and theorems

- We start with one of the two propositions, so either side of $=$ sign
- We transform the expression using axioms and/or theorems, typically by applying one at the time, and stating exactly which axiom/theorem we are using
- Example – prove Th1. $x+x=x$ using axioms only:

$$
\begin{aligned}
x+x &= (x+x)\cdot 1, && \text{Ax2. Neutral element}\\
&= (x+x)(x+x'), && \text{Ax5. Complement of x}\\
&= x+x\cdot x', && \text{Ax4. Distributivity}\\
&= x+0, && \text{Ax5. Complement}\\
&= x, && \text{Ax2. Neutral element}
\end{aligned}
$$

b) Demonstrations using Truth Tables

- In Two valued Boolean algebra theorems are combinatory problems – we can prove given proposition using Truth Tables by checking the values of both sides of expression for all possible combinations of inputs
- Number of variables determine the number of lines in TT
- For each side of the proposition we will have one TT'; thus there will be 2 TTs for a given proposition
- We calculate the results for both expressions and if they are equal we have proved that the proposition is true

b) Demonstration using TTs – Axiom 1

- Set $B = \{0, 1\}$ is closed: in another words the result of three Boolean operators $+$, $\cdot$, $'$, is in B
- Truth tables are given for the operators:

x	x'
0	1
1	0

x	y	$x \cdot y$	$x + y$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

- The results is indeed in B

b) Demonstration using TTs – Axiom 4

Distributivity:

$$x \cdot (y + z) = x \cdot y + x \cdot z$$

x	y	z	$y+z$	$x \cdot (y+z)$	$x \cdot y$	$x \cdot z$	$(x \cdot y) + (x \cdot z)$
0	0	0	0	**0**	0	0	**0**
0	0	1	1	**0**	0	0	**0**
0	1	0	1	**0**	0	0	**0**
0	1	1	1	**0**	0	0	**0**
1	0	0	0	**0**	0	0	**0**
1	0	1	1	**1**	0	1	**1**
1	1	0	1	**1**	1	0	**1**
1	1	1	1	**1**	1	1	**1**

b) Demonstration using TTs – De Morgan's law

$(x+y)' = x' \cdot y'$

x	y	$x+y$	$(x+y)'$	x'	y'	$x' \cdot y'$
0	0	0	**1**	1	1	**1**
0	1	1	**0**	1	0	**0**
1	0	1	**0**	0	1	**0**
1	1	1	**0**	0	0	**0**

$(x \cdot y)' = x' + y'$

x	y	$x \cdot y$	$(x \cdot y)'$	x'	y'	$x' + y'$
0	0	0	**1**	1	1	**1**
0	1	0	**1**	1	0	**1**
1	0	0	**1**	0	1	**1**
1	1	1	**0**	0	0	**0**

c) Duality principal

- In Boolean algebra every result has its dual
- If there is a result S that is true in Boolean algebra, the result $S*$, called dual of S, will be true too
- $S*$ can be obtained by systematically permuting: operators $+$ and $\cdot$ and symbols '0' & '1' in B
- What does this really means is the following: duality principal asserts that Boolean algebra is unchanged when all dual pairs are interchanged
- In another words proving S automatically proves $S*$

c) Duality principal – examples

- Example 1
  - Assume the following result S is true:

$$\forall x, x \in B, x + x = x$$

  - Dual of S, $S*$ is true too:

$$\forall x, x \in B, x \cdot x = x$$

- De Morgan Laws
  - The following is true:

$$(x + y)' = x' \cdot y'$$

  - Dual is true too:

$$(x \cdot y)' = x' + y'$$

c) Duality principle generalized to n variables

- It is possible to generalize the previous result on n variables:

$$\forall x_i, x_i \in B, S(x_1, \ldots, x_n) \rightarrow S*(x_1, \ldots, x_n)$$

- Example – Extension of De Morgan laws on n variables

  If:

$$(x_1 + \ldots + x_n)' = x_1' \cdot \ldots \cdot x_n'$$

  Then:

$$(x_1 \cdot \ldots \cdot x_n)' = x_1' + \ldots + x_n'$$